

Programação em Python

Unidade 3

Desenvolver Aplicações Back-end para Web com Python

Sessão 10

Python

Django - Modelos

- Classe Python baseada na classe Model que permite construir uma tabela de banco de dados a partir de uma estrutura correspondente.
- Utiliza ORM, Mapeador Relacional de Objetos do Django, para realizar operações CRUD.
- Internamente, o motor do ORM de converte a classe/objeto para consultas SQL e vice-versa.



Saiba mais

- Sobre modelos em um projeto Django, visite: <https://docs.djangoproject.com/pt-br/5.1/intro/tutorial02/>
- Mais em: https://tutorial.djangogirls.org/pt/django_models/
- E mais ainda... <https://developer.mozilla.org/pt-BR/docs/Learn/Server-side/Django/Models>

Python

Criando modelos

- Criar uma classe no arquivo `<nome-app>/models.py`:

```
class Aluno(models.Model):  
    matricula = models.AutoField(primaryKey=True, help_text="Informe a matrícula do  
        aluno")  
    nome = models.CharField(max_length=70, null=False, help_text="Informe o nome do  
        aluno")  
  
    def __str__(self):  
        return f'{self.matricula} – {self.nome}'
```

- Preparar a migração utilizando o comando:

```
python manage.py makemigrations
```

- Realizar a migração utilizando o comando:

```
python manage.py migrate
```

Python

Acessando modelos criados

- É possível acessar e manipular os dados dos modelos/tabelas criados.
- Para isso, basta registrar o modelo no App e acessa-lo através do APP “admin”
- Registrar a classe no arquivo `<nome-app>/admin.py`:

```
from aluno.models import Aluno
```



```
# Register your models here.  
admin.site.register(Aluno)
```
- Acessar o modelo/tabela através do APP “admin” no navegador, acessar a URL do servidor remoto:
<http://127.0.0.1:8000/admin>

Python

Definições de Modelos

- O módulo Model do Django possui um conjunto de definições que permitem que seja realizado o mapeamento objeto-relacional.
- É possível criar filtros para retornar um ou mais objetos a partir de uma consulta como se utilizasse o comando WHERE do SQL.
- Podem ser definidos chaves primárias e estrangeiras, onde estas últimas estabelecem os relacionamentos entre as tabelas de um modelo de dados relacional.



Saiba mais

- Sobre modelos em um projeto Django, visite: <https://docs.djangoproject.com/pt-br/5.1/intro/tutorial02/>
- Mais em: https://tutorial.djangogirls.org/pt/django_models/
- E mais ainda... <https://developer.mozilla.org/pt-BR/docs/Learn/Server-side/Django/Models>

Python

Tipos de dados utilizados em definições de Modelos

- **CharField**
 - Permite armazenar textos curtos. É obrigatório especificar o atributo `max_length` (tamanho máximo dos valores a serem armazenados)
- **IntegerField**
 - Permite armazenar valores inteiros e para validar os valores inseridos no formulário.
- **TextField**
 - Permite armazenar textos longos.
- **DateField**
 - Permite armazenar datas. É convertido em um objeto do tipo `datetime.date`.
- **DateTimeField**
 - Permite armazenar a data e a hora. É convertido em um objeto do tipo `datetime.datetime`.
- **EmailField**
 - Permite armazenar e validar um endereço de e-mail.
- **AutoField**
 - Permite gerar um valor automático incremental. É um tipo especial de `IntegerField` e quando não informado o valor assumido é a chave primária da tabela.
- **ForeignKey**
 - Mapeia uma chave estrangeira de um relacionamento “um para muitos” (“one-to-many”).
- **ManyToManyField**
 - Mapeia uma chave estrangeira de um relacionamento “muito-para-muitos” (“many-to-many”).

Python

Atributos utilizados nas definições de tipos de dados de Modelos

- `help_text`
 - fornece um label para o campo no formulário.
- `verbose_name`
 - fornece um nome “humanizado” (human-readable).
- `default`
 - valor padrão para o campo.
- `null`
 - Quando igual a “True”, o campo no banco de dados aceitará o valor Nulo (NULL). O valor padrão é False.
- `blank`
 - Quando igual a “True”, o formulário aceitará “brancos” ou uma string vazia. O valor padrão é False.
- `primary_key`
 - Quando igual a “True”, o campo é considerado como a chave primária da tabela/modelo. Se nenhum campo for definido como `primary_key`, o Django irá criar um campo automaticamente.
- `choices`
 - Permite que o campo recebe um valor a partir de um conjunto de valores e o formulário se transforma em um elemento HTML do tipo `<select>`

Python

Metadados utilizados nas definições de Modelos

- O Django permite informar metadados que trarão comportamentos específicos, por exemplo, às consultas realizadas pelo ORM.
- É definido a partir da implementação da classe Meta

```
# Metadata
class Meta:
    ordering = ['-nome']
```

Obs.:

Neste caso, o resultado da saída será ordenado pelo campo/atributo “nome” do modelo

Python

Métodos utilizados nas definições de Modelos

- O Django permite definir métodos para acessar os objetos montados pelo ORM.
- Obrigatoriamente, o método `__str__(self)`: deverá ser implementado em todos os modelos.

```
def __str__(self):  
    """ Texto que sera utilizado no toString do modelo """  
    return self.nome
```

- Outro método interessante é `get_absolute_url(self)`.
- Ele permite retornar a URL para exibir um único elemento do modelo.
- Ao definir este método, o Django irá automaticamente adicionar um botão “View on Site” na tela de edição do registro na parte administrativa do site.

```
def get_abosolute_url(self):  
    """ Retorna a URL para acesso a uma instancia especifica do modelo Autor """  
    return reverse('model-detail-view', args=[str(self.id)])
```

Python

Exercitando os modelos ORM



Python

Criando o modelo "Tipo de Atividade"

1. Abrir o arquivo "tipodeatividade/models.py"
2. Inserir o código abaixo e salvar

```
1  from django.db import models
2  from django.urls import reverse
3
4  # Create your models here.
5  class TipoDeAtividade(models.Model):
6      """ Modelo representando um Tipo de Atividade """
7      codigo = models.IntegerField(primary_key=True,
8                                  help_text='Código do Tipo de atividade')
9      descricao = models.CharField(max_length=70, null=False,
10                                  help_text='Informe a descrição do Tipo de Atividade')
11
12      def __str__(self):
13          return f'{self.codigo} {self.descricao}'
```

Python

Ativando o modelo “Tipo de Atividade”

1. Abrir o arquivo “escoladosaber/settings.py”
2. Inserir código abaixo na lista “INSTALLED_APPS”

```
1
2  # Application definition
3
4  INSTALLED_APPS = [
5      'django.contrib.admin',
6      'django.contrib.auth',
7      'django.contrib.contenttypes', ...,
8      'tipodeatividade.apps.TipoatividadeConfig',
9  ]
```

Python

Realizando a migração do modelo “Tipo de Atividade”

1. Abrir o terminal
2. Digitar “python manage.py makemigrations tipodeatividade”

Acessando o arquivo gerado para o modelo “Tipo de Atividade”

1. Abrir o terminal
2. Digitar “python manage.py sqlmigrate tipodeatividade 0001
3. Verificar o conteúdo do arquivo

Acessando o arquivo gerado para o modelo “Tipo de Atividade”

1. Abrir o terminal
2. Digitar “python manage.py migrate”

Python

Acessando o modelo “Tipo de Atividade” pelo terminal

1. Abrir o terminal
2. Digitar “python manage.py shell”
3. Digitar os comandos abaixo no shell e verificar a saída apresentada

```
>>> from tipodeatividade.models import TipoDeAtividade
-----
>>> TipoDeAtividade.objects.all()
>>> TipoDeAtividade.objects.create(codigo=1, descricao='Atividade 1')
>>> TipoDeAtividade.objects.create(codigo=2, descricao='Atividade 2')
>>> TipoDeAtividade.objects.all()
>>> TipoDeAtividade.objects.filter(codigo=1)
>>> TipoDeAtividade.objects.filter(descricao__contains='2')
>>> TipoDeAtividade.objects.filter(descricao__contains='2').delete()
>>> TipoDeAtividade.objects.all()
>>> TipoDeAtividade.objects.create(codigo=3, descricao='Atividade 3')
>>> TipoDeAtividade.objects.create(codigo=4, descricao='Atividade 4')
>>> TipoDeAtividade.objects.all()
```

Python

Acessando o modelo “Tipo de Atividade” pelo terminal – continuação

```
>>> TipoDeAtividade.objects.filter(descricao__contains='4')
>>> TipoDeAtividade.objects.filter(descricao__contains='4').update(descricao='Atividade 4 - Alterada')

>>> TipoDeAtividade.objects.all()
>>> TipoDeAtividade.objects.get(codigo=3)
>>> TipoDeAtividade.objects.get(codigo=3).descricao

>>> TipoDeAtividade.objects.get(codigo=3).delete()

>>> TipoDeAtividade.objects.all()
>>> TipoDeAtividade.objects.all().count()
>>> TipoDeAtividade.objects.all().order_by('codigo')
>>> TipoDeAtividade.objects.all().order_by('-codigo')
>>> TipoDeAtividade.objects.all().order_by('descricao')
>>> TipoDeAtividade.objects.all().order_by('-descricao')
>>> TipoDeAtividade.objects.all().values()
>>> TipoDeAtividade.objects.all().values('codigo')
>>> TipoDeAtividade.objects.all().values('codigo', 'descricao')
```

Python

Acessando o modelo “Tipo de Atividade” pelo terminal – continuação

```
>>> TipoDeAtividade.objects.all().values_list()
>>> TipoDeAtividade.objects.all().values_list('codigo')
>>> TipoDeAtividade.objects.all().values_list('codigo', 'descricao')

>>> TipoDeAtividade.objects.all().values_list('codigo', 'descricao', named=True)

>>> TipoDeAtividade.objects.all().values_list('codigo', 'descricao', named=True)[0]

>>> TipoDeAtividade.objects.all().values_list('codigo', 'descricao', named=True)[0].codigo

>>> TipoDeAtividade.objects.all().values_list('codigo', 'descricao', named=True)[0].descricao
```

Python

Criar um usuário para o app “admin”

1. Abrir o terminal
2. Digitar “python manage.py createsuperuser”
3. Informar “admin” para “Username”
4. Informar seu e-mail para “Email address”
5. Informar uma senha para “Password”
 - Sugestão: senha1234
6. Abrir o app “admin” no navegador - <http://127.0.0.1:8000/admin>

Python

Acessando o modelo “Tipo de Atividade” pelo app “admin”

1. Abrir o arquivo “escoladosaber/admin.py”
2. Inserir código abaixo

```
1  ✓ from django.contrib import admin
2    from tipodeatividade.models import TipoDeAtividade
3
4    # Register your models here.
5    admin.site.register(TipoDeAtividade)
6
```

3. Abrir o app “admin” no navegador - <http://127.0.0.1:8000/admin>

Python

- Considerações finais.
- Agradecimentos.



Programação em Python

Botafogo



Até a próxima!

